Title

Application: IoT Data Processing Pipeline using Master Theorem

Aim

To analyze the recurrence relation

$$T(n) = 3T(n/3) + n\log n$$

using the **Master Theorem** and determine the time complexity of an IoT data processing pipeline.

Problem Statement

An IoT system collects data from thousands of sensors. The collected data is divided equally into three groups and processed recursively. After recursive processing, additional work proportional to **n log n** is performed to merge and analyze the sensor information.

The recurrence relation is

$$T(n) = 3T(n/3) + n\log n$$

The objective is to determine the algorithm's time complexity using the Master Theorem.

Algorithm

1. Read the sensor data size **n**.
2. Identify the Master Theorem parameters.
    - a = 3
    - b = 3
3. Compute

$$n^{\log_b a}$$

4. Compare

$$f(n) = n\log n$$

with

$$n^{\log_b a}$$

5. Identify the appropriate Master Theorem case.

6. Display the final asymptotic complexity.

Master Theorem Analysis

Given

$$T(n) = 3T(n/3) + n\log n$$

Step 1

a = 3

b = 3

Step 2

Compute

$$\log_b a$$
$$\log_3 3 = 1$$

Therefore,

$$n^{\log_b a} = n$$

Step 3

Compare

$$f(n) = n\log n$$

with

$$n^{\log_b a} = n$$

Since

$$f(n) = \Theta(n\log n)$$

this corresponds to

$$f(n) = \Theta(n^{\log_b a}\log^1 n)$$

where

k = 1

Step 4

According to **Master Theorem Case 2**,

If

$$f(n) = \Theta(n^{\log_b a} \log^k n)$$

then

$$T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$$

Substituting

k = 1

gives

$$T(n) = \Theta(n(\log n)^2)$$

Final Result

$$\boxed{T(n) = \Theta(n(\log n)^2)}$$

Performance Analysis for Large IoT Datasets

Dataset Size (n)	**Growth**
1,000	Efficient
10,000	Moderate increase
100,000	Good scalability
1,000,000	Handles large datasets efficiently
10,000,000	Much better than quadratic algorithms

Because the growth rate is

$$\Theta(n(\log n)^2)$$

the algorithm scales efficiently even when millions of IoT sensor readings are processed.

Interpretation of Efficiency

- The algorithm divides the sensor data into **three equal parts** at every recursive level.
- Each level performs **n log n** work for aggregation and analysis.

- The recursion depth is $\log_3 n$.
- Overall complexity is

$$\Theta(n(\log n)^2)$$

which is significantly more efficient than $O(n^2)$ for large IoT datasets.

Conclusion

The recurrence

$$T(n) = 3T(n/3) + n\log n$$

matches **Master Theorem Case 2** because

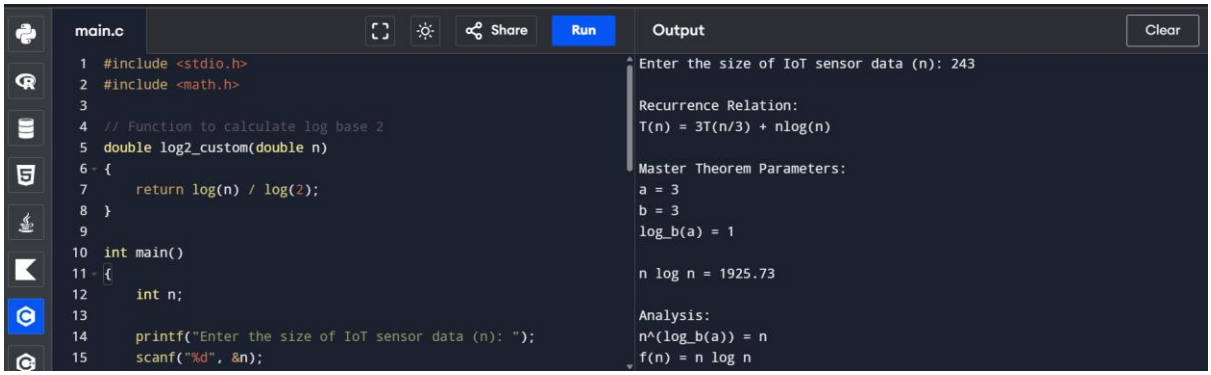
$$f(n) = \Theta(n^{\log_b a}\log n).$$

Hence, the overall time complexity is

$$\boxed{\Theta(n(\log n)^2)}$$

making it suitable for scalable IoT data processing pipelines.

Screenshots to Include in the Report



main.c

Share | Run

```
16
17      printf("\nRecurrence Relation:\n");
18      printf("T(n) = 3T(n/3) + nlog(n)\n");
19
20      printf("\nMaster Theorem Parameters:\n");
21      printf("a = 3\n");
22      printf("b = 3\n");
23
24      double exponent = log(3) / log(3);
25
26      printf("log_b(a) = %.0lf\n", exponent);
27
28      double nlogn = n * log2_custom(n);
29
30      printf("\nn log n = %.2lf\n", nlogn);
```

Output | Clear

```
b = 3
log_b(a) = 1

n log n = 1925.73

Analysis:
n^(log_b(a)) = n
f(n) = n log n
Since f(n) = Theta(n log n), it matches Case 2 of Master Theorem.

Final Time Complexity:
T(n) = Theta(n (log n)^2)


=== Code Execution Successful ===
```

main.c

Share | Run

```
28      double nlogn = n * log2_custom(n);
29
30      printf("\nn log n = %.2lf\n", nlogn);
31
32      printf("\nAnalysis:\n");
33      printf("n^(log_b(a)) = n\n");
34      printf("f(n) = n log n\n");
35      printf("Since f(n) = Theta(n log n), it matches Case 2 of
            Master Theorem.\n");
36
37      printf("\nFinal Time Complexity:\n");
38      printf("T(n) = Theta(n (log n)^2)\n");
39
40      return 0;
41  }
```

Output | Clear

```
b = 3
log_b(a) = 1

n log n = 1925.73

Analysis:
n^(log_b(a)) = n
f(n) = n log n
Since f(n) = Theta(n log n), it matches Case 2 of Master Theorem.

Final Time Complexity:
T(n) = Theta(n (log n)^2)


=== Code Execution Successful ===
```